1. Descripción Breve del Sistema y del Módulo MVP

- Sistema General (SHOP): Plataforma de comercio electrónico desarrollada con un backend en NestJS, TypeORM y PostgreSQL, y un frontend SPA en React, Vite, TypeScript y Zustand. Permite la gestión de catálogos con paginación y filtros, administración de productos, subida segura de imágenes y mensajería en tiempo real mediante WebSockets.
- Módulo MVP (Auth): Encargado de la identidad, control de acceso y vigencia de sesiones de usuario. En su estado actual totalmente refactorizado, comprende las rutas de registro (POST /api/auth/register), inicio de sesión (POST /api/auth/login), renovación de estado (GET /api/auth/check-status) y cierre de sesión (POST /api/auth/logout). La autenticación opera enteramente mediante un JWT almacenado en una cookie httpOnly (sin exposición en el cuerpo de respuesta ni en localStorage), complementado con un mecanismo robusto de protección contra falsificación de solicitudes entre sitios (CSRF) mediante un token firmado gestionado por CsrfGuard y TokenService. La autorización se rige estrictamente por roles (admin, super-user, user).
-

2. Vista Lógica Final del Sistema

La arquitectura del sistema y del módulo Auth está desacoplada en capas estrictas con responsabilidades delimitadas:

- Capa de Presentación:
    - Frontend (React + Zustand): Gestiona las vistas de acceso (LoginPage, RegisterPage), el estado global de sesión mediante useAuthStore (el cual opera totalmente desprovisto de tokens o de localStorage) y la protección de rutas mediante componentes dedicados (ProtectedRoutes). El cliente HTTP de Axios (eShopApi.ts) activa withCredentials: true e incluye automáticamente la cabecera X-CSRF-Token en solicitudes de mutación (POST, PUT, PATCH, DELETE).
    - Backend (AuthController y DTOs): Expone la API REST bajo el prefijo /api. Los datos de entrada son validados por DTOs estrictos (CreateUserDto, LoginUserDto), y las respuestas públicas serializan la entidad omitiendo datos sensibles mediante DTOs de salida con listas blancas basadas en @Expose() (UserResponseDto, AuthResponseDto).
- Capa de Control de Acceso:
    - Decoradores y Guards (@Auth, @RoleProtected, @GetUser, UserRoleGuard, CsrfGuard): El decorador compuesto @Auth() unifica la validación de autenticación de Passport, la verificación de roles mediante UserRoleGuard y la exigencia del token CSRF a través de CsrfGuard en operaciones que modifican estado.
    - Estrategia Passport (JwtStrategy): Extrae de manera transparente y valida el JWT directamente desde la cookie access_token, verificando la firma, la vigencia y el estado activo del usuario en la base de datos.
- Capa de Aplicación:

    - AuthService y TokenService: AuthService concentra la lógica de negocio (cifrado con bcrypt, validación de credenciales, manejo estandarizado de excepciones y errores de base de datos). TokenService centraliza la emisión segura de cookies HTTP (access_token y csrf_token), los parámetros de expiración y la validación del doble token CSRF firmado.
- Capa de Persistencia:
    - UserEntity + Repository<User>: Representación relacional mapeada mediante TypeORM sobre PostgreSQL (esquema de usuarios con campos de identificación, correos normalizados, contraseña declarada con select: false, que evita cargarla por defecto en las consultas, y roles).

3. Descripción de los Diagramas C4 del Módulo MVP

Nivel 1: Contexto del Sistema

- Actores: Cliente y Administrador.
- Sistema Central: Plataforma SHOP (E-commerce).
- Interacción: El módulo Auth actúa de manera centralizada en la plataforma determinando la identidad y autorizaciones de los usuarios. Los clientes interactúan para autenticarse y navegar el catálogo; los administradores poseen privilegios adicionales para gestionar productos y subir archivos multimedia al servidor. No intervienen sistemas externos de autenticación de terceros.

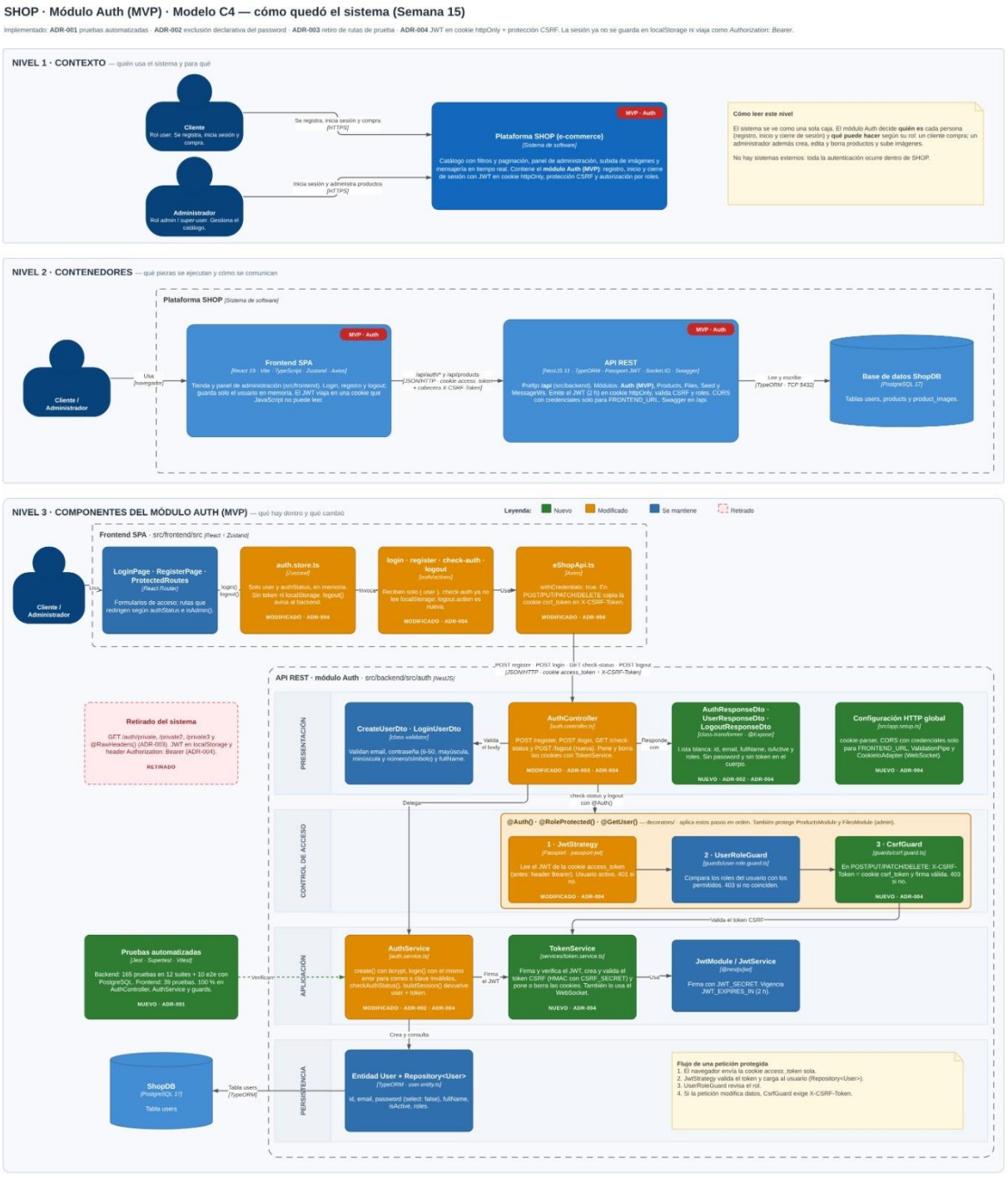
Nivel 2: Contenedores

- Frontend SPA (React + Vite + Zustand): Aplicación cliente de ejecución en navegador que maneja la interfaz de usuario, catálogos, paneles de administración y flujos de autenticación sin almacenar credenciales en el almacenamiento local. Se comunica con el backend mediante peticiones HTTP asíncronas con envío automático de credenciales de cookies.
- API REST (NestJS + TypeORM + WebSockets): Servidor backend central estructurado para procesar la lógica de negocio, emitir cookies de sesión firmadas, validar protecciones CSRF, gestionar el acceso a la base de datos y proveer canales de comunicación en tiempo real autenticados con la cookie access_token que el navegador envía en el handshake de WebSocket.
- Base de Datos ShopDB (PostgreSQL 17): Motor de base de datos relacional persistente que almacena las tablas del sistema (users, products, product_images) operando bajo esquemas seguros gestionados por TypeORM.

Nivel 3: Componentes del Módulo Auth (Frontend y Backend)

- Componentes del Frontend:
    - LoginPage / RegisterPage / ProtectedRoutes: Páginas de acceso y componentes de redirección lógica condicionados por el estado de sesión de Zustand.
    - auth.store.ts: Administrador del estado global de sesión (user, authStatus), sin conservar el token ni usar localStorage.

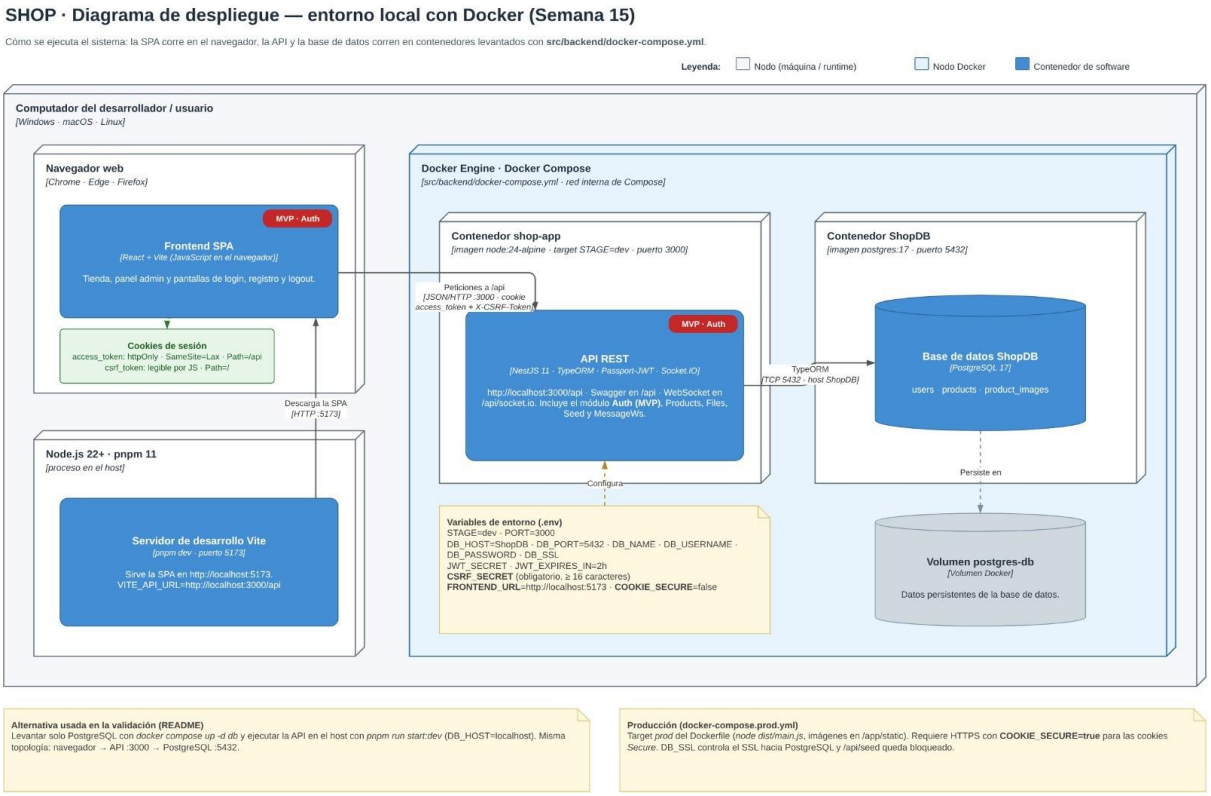
    - login.action / register.action / check-auth.action / logout.action: Servicios de comunicación HTTP orientados a endpoints de autenticación.
    - eShopApi.ts: Instancia global de Axios configurada con withCredentials: true e interceptores para inyectar la cabecera X-CSRF-Token en métodos de mutación.
- Componentes del Backend:
    - AuthController (Presentación): Controlador encargado exclusivamente de recibir peticiones en los endpoints limpios de registro, login, verificación y cierre de sesión.
    - CreateUserDto / LoginUserDto (Presentación): Clases validadoras de formato, seguridad y longitud de credenciales mediante class-validator.
    - AuthResponseDto / UserResponseDto (Presentación): DTOs de salida enriquecidos con @Expose() de class-transformer para garantizar una lista blanca segura en la respuesta.
    - TokenService (Aplicación): Componente especializado en la firma y verificación del JWT, la emisión y eliminación de las cookies de sesión (access_token y csrf_token) y la generación y validación del token CSRF firmado con HMAC.
    - CsrfGuard & UserRoleGuard (Control de Acceso): Guards encargados de la validación estricta del doble submit de tokens CSRF en operaciones de modificación y de la verificación de privilegios por roles.
    - JwtStrategy (Control de Acceso): Estrategia de Passport configurada para extraer el token JWT directamente de la cookie segura access_token.
    - AuthService (Aplicación): Núcleo de la lógica de negocio: cifrado de contraseñas con bcrypt, validación de credenciales, control de excepciones de duplicidad (código 23505 de PostgreSQL) y armado de la respuesta pública mediante UserResponseDto.
    - UserEntity + Repository<User> (Persistencia): Entidad de TypeORM; la columna password se declara con select: false y la exclusión en las respuestas la resuelve UserResponseDto.
    - Suite de Pruebas Automatizadas: Conjunto integral compuesto por 165 pruebas en 12 suites de Jest (unitarias y de integración con cobertura de sentencias del 92.33% en el módulo Auth y del 100% en AuthController, AuthService y CsrfGuard), 10 pruebas e2e conectadas a PostgreSQL, y 39 pruebas en el Frontend con Vitest.



4. Descripción del Diagrama de Despliegue (Entorno Local y Producción)

- Entorno del Host de Desarrollo:
  - Navegador Web del Usuario: Entorno cliente seguro que procesa la SPA ejecutando los scripts de la interfaz.
  - Servidor de Desarrollo Vite (Node.js 22 + pnpm): Proceso ejecutado en el puerto local 5173 (pnpm dev) encargado de servir la aplicación frontend; la SPA llama directamente a la API en VITE_API_URL (http://localhost:3000/api).
- Entorno de Contenedores (Docker Engine + Docker Compose):

- Red interna de aislamiento gestionada mediante docker-compose.yml.
- Contenedor shop-app: Contenedor basado en imagen Node.js Alpine que ejecuta la API de NestJS en el puerto 3000 con las variables de entorno requeridas (STAGE=dev, PORT, conexión a la base de datos, JWT_SECRET, CSRF_SECRET (obligatoria, de al menos 16 caracteres), FRONTEND_URL, COOKIE_SECURE, etc.).
- Contenedor ShopDB: Contenedor de PostgreSQL 17 con el puerto 5432 publicado también en el host.
- Volumen Docker postgres-db: Sistema de almacenamiento persistente montado para preservar los datos relacionales ante reinicios del contenedor de base de datos.



5. Refactorización Realizada

5.1. Problemas de la Versión Inicial

1. Exposición de rutas de prueba innecesarias: Presencia en producción de los endpoints GET /auth/private, /private2 y /private3, los cuales exponían información de cabeceras internas (@RawHeaders) y trazas de depuración.
2. Manipulación manual e insegura de datos sensibles: Eliminación manual de la contraseña en los servicios mediante delete user.password, lo que representaba un riesgo latente de filtración ante la creación de nuevos endpoints.
3. Almacenamiento vulnerable del JWT: Dependencia de localStorage para guardar el token de autenticación, exponiendo el sistema a ataques de scripts cruzados (XSS).
4. Carencia total de pruebas automatizadas: Ausencia de pruebas de regresión: el módulo partía de cero pruebas automatizadas, sin forma de verificar los cambios automáticamente.

5.2. Cambios Aplicados en el Diseño y el Código

- Migración a Cookies httpOnly y Doble Protección CSRF (ADR-004): Se rediseñó el flujo de sesión reemplazando el almacenamiento local por la emisión de dos cookies gestionadas por TokenService: access_token (configurada como HttpOnly, SameSite=Lax y Path=/api, con la misma vigencia del JWT y con el atributo Secure cuando COOKIE_SECURE=true, lo que en producción exige servir la aplicación por HTTPS) y csrf_token (legible por JavaScript para poder reenviarla en la cabecera). Se implementó el CsrfGuard para exigir y validar criptográficamente la cabecera X-CSRF-Token frente a cualquier mutación de estado (POST, PUT, PATCH, DELETE). Respecto al diseño previsto, el CSRF se resolvió con un guard (CsrfGuard) y un servicio (TokenService) en lugar de un middleware; la cookie csrf_token usa Path=/ porque con Path=/api el navegador no la entregaría al JavaScript de las páginas de la SPA; y el WebSocket pasó a la ruta /api/socket.io para que el navegador envíe la cookie access_token en el handshake.
- Exclusión Declarativa de Credenciales (ADR-002): Se eliminó el uso de delete user.password reemplazándolo por DTOs de salida con lista blanca (@Expose(): UserResponseDto y AuthResponseDto), que AuthService construye en un único punto; además, la columna password conserva select: false en la entidad. Frente a lo planeado (@Exclude() en la entidad con un interceptor global de serialización), se optó por la lista blanca porque una columna sensible nueva queda excluida por defecto, en lugar de depender de marcarla manualmente.
- Retiro Definitivo de Endpoints Obsoletos (ADR-003): Se eliminaron por completo las rutas /private* del controlador y el decorador @RawHeaders, acotando la superficie de ataque del sistema y estandarizando el control de acceso únicamente mediante el decorador unificado @Auth().
- Implementación Integral de Pruebas Automatizadas (ADR-001): Se construyó una infraestructura robusta con 165 pruebas unitarias y de integración en el backend (distribuidas en 12 suites con Jest, que cubren el flujo HTTP de autenticación, la estrategia de Passport, los guards de roles y CSRF, el servicio de tokens, el gateway de WebSocket, el controlador de archivos y los servicios de productos y de seed), 10 pruebas e2e ejecutadas contra una base de datos PostgreSQL, y 39 pruebas desarrolladas en el frontend mediante Vitest, alcanzando una cobertura de sentencias del 92.33% en el módulo Auth.
- Endurecimientos y Ajustes Estructurales Adicionales: Se incorporó el bloqueo automático del endpoint de seed (GET /seed) al operar en entornos de producción (STAGE=prod), se restringió la subida de archivos exclusivamente a perfiles con rol de administrador, las respuestas públicas de productos dejaron de incluir los datos del usuario creador, se unificó el mensaje del error 401 del inicio de sesión para no revelar si el correo existe, y se incorporaron nuevas variables de entorno (CSRF_SECRET, obligatoria y de al menos 16 caracteres; FRONTEND_URL, COOKIE_SECURE, JWT_EXPIRES_IN y DB_SSL, opcionales).

5.3. Mejora de Atributos de Calidad Priorizados

Atributo de Calidad	Justificación de la Mejora Medible Basada en la Refactorización

Seguridad	Reducción del impacto de un XSS: al guardar el JWT en una cookie httpOnly, un script malicioso del lado del cliente ya no puede leerlo (httpOnly no impide por sí solo otras acciones de ese script, por lo que se mantiene la validación CSRF). Protección contra ataques CSRF mediante la validación, en CsrfGuard, de un token firmado con HMAC en cada operación de mutación de estado. Eliminación de la superficie de ataque expuesta por las rutas de prueba /private*. Exclusión de los hashes de contraseña en las respuestas mediante DTOs con lista blanca (@Expose). Verificado con pruebas de integración y e2e (401 sin sesión válida, 403 sin token CSRF válido) y con una ejecución real de la API.
Mantenibilidad	Desacoplamiento de la lógica de transformación de datos mediante DTOs declarativos y listas blancas (@Expose), eliminando la necesidad de operaciones manuales propensas a errores humanos en los servicios de negocio. Centralización de la gestión de tokens y cookies en un componente especializado (TokenService), lo que simplifica futuras modificaciones en las políticas de sesión.
Testabilidad	Incorporación de una red de pruebas automatizadas masiva (165 pruebas en el backend, 10 e2e y 39 en el frontend) que permite validar de forma continua y automática los flujos críticos de autenticación, autorización por roles e integridad de datos ante cualquier cambio en el código fuente, alcanzando una cobertura de sentencias del 92.33% en el módulo Auth.